# AGENTIC TOOL DETECTION

Framework for Clawesque detection and management

## Summary

The new class of autonomous AI agent systems are progressively becoming relevant and business class from grassroots projects; many platforms have shared the spotlight of supportable Agentic systems that are far more capable than chats and standalone agents. While the names will change, the platforms which govern them will need a better source of truth to maintain enterprise observability, whether embraced or not by the business, the ability to effectively manage these systems is critical in the path to adoption for applicable ATT use cases

Hermes, Open claw, Nemo Claw or systems alike, the common properties they all share need to be further dissected and envisioned to protect, adopt, or converge with existing policies. The foundational framework shares similar properties to agents as they relate agentic overlay project, however how they operate can introduce new opportunities to provide telemetry into the mechanics of these advancements.

Autonomy, persistence, broad tool access reaching external services, map directly into known and unknown territories for advanced management opportunities and risks alike from a SOC's POV, a long-lived autonomous process with internet egress and local code execution capabilities that are unmonitored or gone unchecked.

Key Recommendations

- Classify the category, not the brand – Build detection and policy around shared behaviors, not around a binary name
- Assume agents are already running – Run a discovery sweep before anything else; shadow agents are the highest likelihood with the lowest visibility
- Mediate egress points – force model provider and messaging traffic through an inspecting proxy with a default-deny allowlist
- Provide a sanctioned path – a hardened, observable agentic system removes the incentive to run shadow agents and focuses the risk where it can be governed

## The Agentic Loop

OpenClaw, NemoClaw, Hermes style forks are not chat assistants embedded in a product, they are general purpose operators where a single process runs a perception reasoning loop that calls a language model, in simplified terms, which can constitute a Local language model, large language model and compatible variants. The agentic loop then acts on the model's decision through a set of tools, and repeats. Around the loop sit several capability surfaces that distinguish this class from ordinary software and help define its governance profile.


*Figure 1- Autonomous agent loop*

## Shared Architecture

- Model provider egress – The agent reaches out to the internet if using LLM commonly OpenRouter, nVidia NIM, a vendor portal or a self-hosted endpoint
- Tool Execution – Tools include Shell, filesystem read/write, HTTP, and code execution. Theis a local code execution by design
- MCP Tool Servers – The MCP lets the agent load additional tools dynamically based on the request or action needed. Loaded using a local 'sidecar' approach appended later after installation.

- Messaging Gateways – A single messaging gateway bridges the agent to mobile apps allowing direct communications, data egress and ingestion to and from the operator. Common Messaging platforms include Telegram, Discord, Slack, WhatsApp and Signal.
- Cron Scheduler – Built-in scheduling runs tasks unattended on a recurring basis.
- Persistent Memory – Memory build on markdown files which persist across sessions and shape future behavior influenced by the operator.
- Self-Modifying skills – The agent creates and refines its own skills during use, sometimes pulled from public skills registries. Considered a live software supply chain surface inside of a running process.
- Sub-agent spawning – The agent can spawn isolated child agents for parallel, specialty work

### Solving the variant problem

While the varying providers are developing, the numerous forks, design changes, improvements, degradation and other versioning actions will continue to proliferate; the loop capabilities remain constant for operations to work. Detection methods that target a specific vendor brand, version or other moving target will not remain as constant as the behavior that is detected.

Agentic Risks

| Risk | Description | |
|---|---|---|
| **Excessive Agency** | Agent takes consequential actions (delete, transfer, send ) beyond intent, amplified by chained tool calls | ATLAS Execution/Impact |
| **Covert Egress** | Messaging gateways behave like a command control channel.<br>Model egress can carry exfiltrated data | Exfiltration over web/ Messaging service |
| **Memory Poisoning** | Malicious entries in memory state steer future decisions across sessions | Data/model poisoning |
| **Skill supply chain** | Registry sourced skills introduce unreviewed code into privileged processes | Compromised supply chain risk |
| **Prompt injection** | Manipulated content in fetched pages, files, or tool output compromises the agents objective | LLM Prompt injection |
| **Credential sprawl** | API keys and platform tokens stored in plaintext configs (nested .env) can be read or exfiltrated | Credential access; Prompt injection |

| | | |
|---|---|---|
| **Subagent sprawl** | Uncapped child agents multiply tool actions, obscure egress, run as subprocess | Excessive agency: resource jacking |
| **Cron persistence** | Scheduled tasks continue unless fully decommissioned, restarts dead actions | persistence |
| **Shadow deployment** | Unsanctioned instances running on endpoints with no monitoring or governance | Shadow IT; unauthorized tooling |

### Priority concerns

- Mediated, not blocked – Model egress and messaging are core agent functions, the control objective inspection and allow-listing at chokepoint
- Human is weakest link in the loop – Any agent that browses or ingests untrusted data must be treated as potential advisory influenced on every turn
- Persistence in memory - Outlives sessions, remediations that kill a process but leave traces such as crons, memory files and stored keys intact does not remove the agent directives

### Opportunities for Sandbox controls

As part of my research, introduced using a Git repo, which is a practice that I have in place to sanitize the memory files and create a uniform configuration and detect drift, has yielded positive results.

1. Create a standardized runtime image using docker (as tested), a configuration baseline with security controls, openclaw.json , allow/deny list, prompt injection training.
2. Create the agent memory file master with strictly enforced rules as part of image
3. Sync Local repository to Gihub repo using GUID based naming conventions
4. Run jobs in Git/Code to periodically detect and remediate drift, poor content management, scrub token and keys to keep poor hygiene under control.
5. Standardize MCP/Skill repository with pre-selected resources
6. Schedule remote clean-up crons without associated content or current in near real-time.
7. Served over iframe in web console experience, driving a single proxy egress on host server
8. Auto-provisioned tenant image with managed configuration in fully isolated dedicated persistent user experience
9. When session is created, it auto provisions a GUID named user repo branch that has a PAT token with least privilege configuration and synchronized the users agent md operating files

Consider this a managed configuration with alerts, automation and metrics to prevent drifting and unintentional security risks before hitting the network. Adopted from Intune management principles for VDI environments and CI/CD pipelines.

### Reference Architecture

The architecture separates an untrusted external tier, consisting of model providers, messaging platforms, public skill and MCP sources, from the corporate environment by a controlled egress layer that every agent relevant flow must traverse. Inside the corporate environment, sanctioned agents run only with a hardened enclave, anywhere else, an agent is considered unsanctioned and is a detection target.

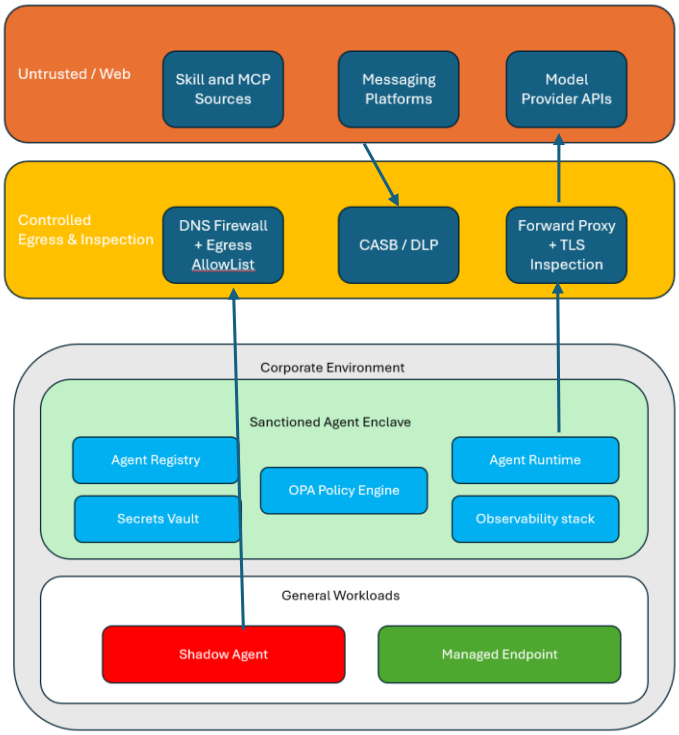



*Figure 2 - Detectable reference framework*

## Design Principles

- Assume breach – Design as if any single agent will be compromised or prompt injected
- Mediate Egress – No agent reaches a provider except through the inspecting proxy and DNS allowlist
- Per-Agent Identity – Every agent has its own machine identity and scoped credentials. Never shared keys, never a human's token
- Default-deny tools – Tool and MCP grants are explicit allow list, evaluated by policy at call time
- Least privilege isolation – agents run in rootless, network restricted sandboxes with no host mount points no standing access to data they do not need

- Full auditability – Every model call, and policy decision is logged to tamper-evident storage. An un-logged action is a control failure.

## Detection

The first operational question: Are these agents running here?

Installation pre-requisites are easier to meet when the information is available to standard users, especially users that already have a MAC platform to use. Installation requires no privileged software, and egress is dependent on HTTPS (commonly port 443), which makes shadow agents easy to stand up and difficult to notice.

Detection combines host indicators, network indicators, and behavioral analytics built into SIEM and mapped to response actions.

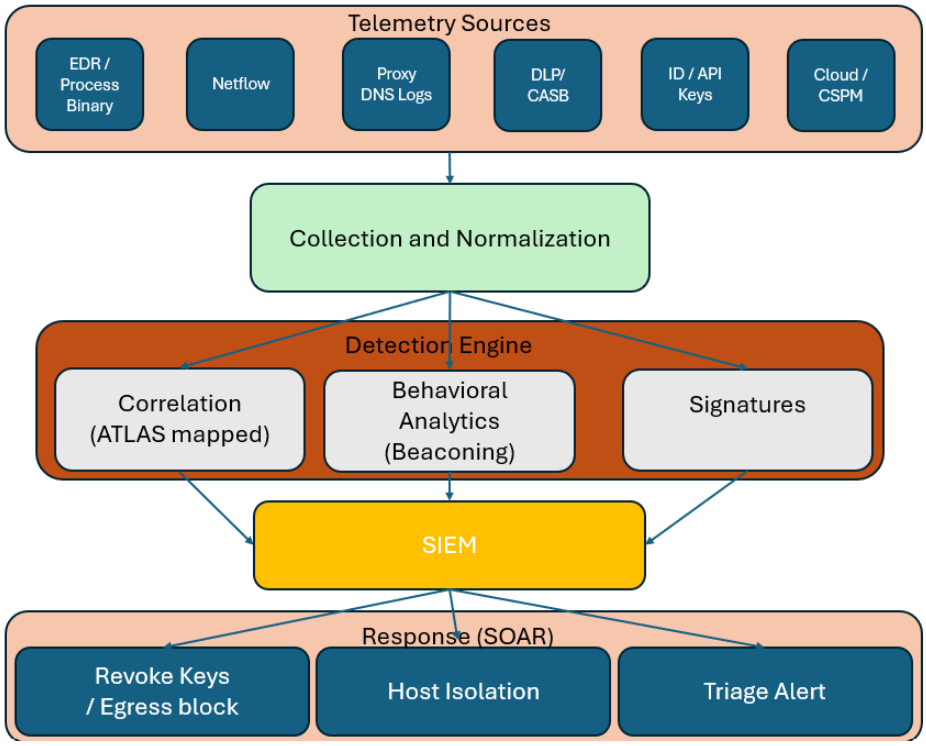



*Figure 3 - Detection pipeline*

## Host indicators

- Install and state paths – user scoped directories such as ~/.openclaw and ~./Hermes and persona memory files (SOUL.md, MEMORY.md, USER.md) , %LOCALAPPDATA%\hermes
- Process and runtime indicators – Python processes launched form a uv-managed (uv init)) initialized sessions in .venv gateway. A gateway maintaining outbound websockets. Portable Git (MinGit)
- Dependency traces – Installation of ribgrep, ffmpeg, and node.js bundled with Python runtime in user profile consistent with the documented installer.

## Network Indicators

- Model provider egress – Repeated API calls to LLM aggregators and provider endpoints without sanctioned business use case associated with them. Services such as OpenRouter, nVidia NIM, harness vendor portals and similar.
- Massaging beacons – Persistent connections to Telegram, Discord, Slack, or Signal endpoints. The single biggest indicator from a host of an agent control channel.
- Local MCP Ports – Loopback listers in non-standard ports such as sidecar, accompanied by outbound tool traffic.
- Off-Hours activity - regular outbound bursts, persistent connections keep-alive, consistent with cron scheduled actions used to run on scheduled time or interval pattern.

## Behavioral indicators

- Autonomous off-hours activity - File, network, processes activity without logged interactive login.
- Tool call bursts and fan-out – Rapid sequences of distinct child processes, API calls indicating sub-agent sprawl.
- Self-modification – A process writing executable scripts into its own skills/config directory and then invoking them

**Commented [IA1]:** SANDBOX USE CASE: Need to sniff the settings or changes to validate for more references. Check product documentation

**Commented [IA2]:** SANDBOX USE CASE: Need to validate with wireshark, or related network tools with vanilla build to determine gateway, ports, websites that are operationally connected both in default configuration and when defalut services are enabled.

| Layer | Signal | Notes |
|---|---|---|
| **Host** | ~/.oenclaw, ~/.hermes, %LOCALAPPDATA/hermes | Install state directories |
| **Host** | Python form uv env | Virtual environment dependencies |
| **Host** | MinGit under install path (Windows) | MinGit – Portable Git used to run shell commands |
| **Host** | Ripgrep, ffmpeg, node co-installed on user profile | Installer dependency footprint |
| **Network** | Persistent Telegram, discord,slack, signal | Server→non-interactive host=control channel |
| **Network** | Egress to LLM aggregators/providers | OpenRouter, NIM |
| **Network** | Loopback MCP Listener + outbound tool traffic | Sidecar pattern |
| **Behavior** | Fixed interval off-hour burst patterns | Cron scheduler running unattended |
| **Behavior** | Process fan-out with no interactive login | Subagent spawning / autonomous action |
| **Behavior** | Process writing then executing own scripts | Self-modifying skills |

## Detection Engineering

Based on the categories, the approach to modern AI can often lead to unintended consequences due to misconfiguration, shadow AI and overall unmitigated practices. The varying levels of monitoring should not be reactive; it should pave the path to enriched process governance and approved channels for AI productivity.

At the configuration level, some guidance to effective personal agentic workflow guardrails must account for most secure known methods to allow for proper alignment to workflows, dedicated resources and the knowledge of how these areas can become more secure, defined and dynamic enough to account for the opportunistic adversarial approaches.

| Finding | Mitigation | Notes |
|---|---|---|
| **Auth Mode = 'none'** | Never use auth.mode.none on loopback paths (Tailnet) | |
| **egress data transfer** | Egress allowlist, prompt-level redaction/DLP | |

| | | |
|---|---|---|
| **Break-glass flags, allowinsecureAuth, dangerouslyDisableDeviceAuth** | forbid in prod, detect as config drift | |
| **Skills /MCP supply chain** | Pin approved skills, restrict tool lanes | |
| **Model calls not traceable** | API monitors, request-level wrappers | |
| **Agents act on untrusted content, indirect prompt injection through tools/exec calls** | input provenance, agent run traces | Instruction data separation, response and content scanning, HITL escalation points, least privilege tool lanes |
| **poisoned memory persistence across sessions** | memory write audit, retrieval source logs | Source allow lists, session isolation |
| **authentication silently disabled (auth.mode=none)** | auth-mode and config change events, token less connection logs | Forbid auth mode changes, config drift alerts, SSO+MFA Reverse proxy |
| **execution surface (shell, browser, node)** | Exec logs | |
| **Multi-agent trust, privilege escalation** | pre-agent egress proxy, pre-agent scoped tokens | Least privileged |
| **Supply chain - tool/MCP poisoning, post approval rug pull** | Skill/MCP inventory + Hashes, openclaw security audit | pin and review before install, re-audit on change, provenance tracking |
| **Sensitive data egress to model providers or through tool calls** | DLP, egress metadata | egress allowlist, local llm for sensitive data, key management vault |
| **invisible per-model calls** | per-request wrappers, Prometheus | ship log traces, |

## Agentic Overlay Integrations

The agentic overlay must assume that any agent with code execution, and feedback loops can become an accelerator for harmful iteration if the tool scope is too broad. Personal Agent controls should focus on sandboxing, tool allowlist, approval gates, egress controls, and audit trails around code execution and external feedback loops.

| Vulnerability | Security Issue | AO – Relationship | |
|---|---|---|---|
| AI -Assisted EDR Evasion development workflow | AI Development loop | The AO must assume that any agent with code execution and feedback loops can become an accelerator for harmful iteration. | |
| Red-Team framing | Configured roles for Red-Team style vulnerability mockup | Masked sub-tasks in sub-agent roles across multiple | AO Should track parent-child task lineage, context, tool access, and completions. Sub-agents should stay isolated by default, with restricted tools and no automatic privilege escalation |
| MCP/Git integration creates agent→Tool execution path | MCP/tool bridges can convert model output into durable state changes using commits, tickets, code updates, task claims, or tool invoke. | Tool calls should be sealed/fixed-schema where possible not HTTP or shell access | Personal agents should be treated as a control-plane boundary tracking tool installation, tool definition changes, credential scope changes, repo actions, should be human gates and auditable |

| | | | |
|---|---|---|---|
| Counter-defensive automated research ingestion | Agent orchestrated playbook defines agentic capabilities and extensibility utilizing Intake, analyze, test, and report methodologies | Risk identifies a chain that can itemize and exfiltrate local/network data points using a research pattern | Approval gates between analyze→lab test, Lab Test→report. Research ingestion should be allowed, however execution against real systems should require explicit authorization, sandbox and telemetry. |
| Repeatable tasks in lab configuration, testing EDR presence, capabilities of remote host | Bounded workflows | Approved targets only allowed, no production targets, captured telemetry. | XSIAM/Cribl, detect correlated tool executions, endpoint actions processes |
| Python generated payloads | Rust ang GO generated payloads wrapped in encryption/evasion methods | Modular generation enables rapid variant creation and testing. | Treat variant generation workflow, scripts, code-gen, test harness, prompt driven build loops as high risk when connected to execution. Restrict with sandboxing, human approval and EDR/XDR monitoring. |
| Agentic misinterpretation/reporting | Agent-related reports can overstate success, misread results, or create false confidence. | AO reporting should require evidence of artifacts such as logs, screenshots, | Arize or equivalent observability should track model claims against evidence. |
| Decision tree automation in place of LLM | Decision tree automation can seem agentic, even without a reasoning LLM | AO should monitor deterministic orchestrators, queues, cron | SNOW/Cribl/XSIAM should include non-LLM automations |

| | | | |
|---|---|---|---|
| | | jobs, task runners and workflow engines not just LLM calls | |
| Adversarial traffic differentiation | Beacon traffic patterns and services utilized need to be baselined to determine the traffic anomalies and irregularities | API's, AgentMail, Telegram, model calls are typical services utilized by personal agents, determine to filter, restrict or choose a path for use case. | XSIAM/Cribl to help distinguish normal agent traffic from unusual destinations, methods or payload sizes |
| Shellcode injection scripts | Python-based malware development scripts injecting shellcode into legitimate Windows executables | Watch unexpected child processes from agent runtimes, unsigned binaries, sensitive file access attempts | EDR process tree monitoring |
| Kill-chain tools flows | AI orchestration across known tooling | Map tool permission to risk categories: Discovery, collection, credential access, privilege escalation, defense evasion, lateral movement | |
| Agent-generated documentation and scoring | Automated documentation can appear as polished and true, however | Reports should preserve provenance, a clear explanation of | Append the 5 W's and H to each doc, with supportive tools, permissions |

| | | | |
|---|---|---|---|
| | provenance should be determined | the purpose and detailed evidence of conclusions | |
| | | | |

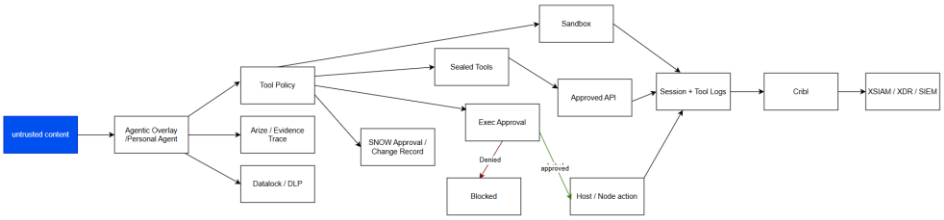
## Mapping services to role requirements

| Service | AO Role | Finding | Notes |
|---|---|---|---|
| DataLock | Data loss/ sensitive content | Exfiltration / credential exposure | Redaction, DLP, sensitive-data movement detections |
| PrismaAIRS | Runtime posture / security governance | Prompt injection / risky agent workflows / AI app posture | Risk assessment and guardrail validation |
| LiteLLM | Model gateway / provider routing | Control plane and routing visibility | Useful only if not bypassed |
| XSIAM | XDR/SIEM correlation | EDR evasion, process tree monitoring, lateral movement | Central detection/correlation |
| Arize | LLM observability/ Evals | Agent overclaiming / trace review, prompt/tool behavior drift | Must avoid logging sensitive data / secrets |
| Watsonx | Enterprise AI governance/model | Governance and control plane | Evaluate overlap in scope/policy as related to personal agents |
| Astra | Data/API integration | Data access boundary and credential scope | Database/vector/data service and required permissions |
| ServiceNow | Approval/Change control | Human-gated approvals, provenance | Ticketed authorization change records |
| A365 | M365/Api integration | Enterprise productivity data | Defined Graph/API scopes r/w permissions |
| Cribl | Telemetry pipeline routing | XDR/SIEM routing, agent/process/log normalization | XSIAM personal agent telemetry and service integration logs |

| Service | Operation |
|---|---|
| Datalock → | DLP/Data Boundary |
| Prisma AIRS → | AI Posture/Guardrail Validation |
| LiteLLM → | Model Gateway / Routing |
| XSIAM/CRIBL → | Telemetry/XDR |
| Arize → | Trace / Evidence Validation |
| WatsonX → | Enterprise AI Governance |
| Astra/A365→ | Data/API Boundary |
| ServiceNow→ | Human Approval Control |

## Architecture diagram

Workflow 1 – Data flow for agentic action workflow / approval gates / closed loop




Service queue mapping

### Project Testing Artifacts

- JIRA Story

1. ARCH-xx Define the AT&T Agentic overlay control plane
   Category: Architecture/Governance

As an enterprise architect, I want a defined agentic overlay control plane so that AI agents can coordinate work across ATT systems without bypassing existing governance, identity and operational controls.

The agentic overlay should sit above existing systems as an orchestration layer, not replace them. It should route tasks, enforce policy, manage agent permissions, capture evidence and determine when human approval is needed.

Acceptance criteria

- Agentic overlay components are documented: agents, tools, policy gates, memory, workflow engine and evidence store
- Existing AT&T Systems remain systems of record
- Agents access systems only through approved APIs or controlled tools
- HITL approval is required for high-impact actions
- Every agent action is logged with actor, source, decision and outcome

2. ARCH-xx – Establish Identity bound agent execution
   Category: Architecture/Security

As a security architect, I want every agent action bound to an approved identity so that autonomous execution remains traceable, auditable, and compliant with AT&T enterprise security standards

Description

Agents should not act as anonymous automation. Each agent, workflow, and tool call should use scoped service identities, delegated user context where appropriate, and policy-based access controls.

Acceptance criteria

- Each agent has a defined service identity or delegate execution model.
- Tool access is scoped by role, workflow and risk level
- Privileged actions require policy validation before execution
- Agent actions are traceable to initiating user, workflow and approval state
- Unauthorized Identity elevation attempts are blocked and logged

3. ARCH-xx – Create evidence first architecture for agentic workflows
   Category: Architecture/ Auditability

As an enterprise governance owner, I want agentic workflows to produce evidence by default so that ATT can validate, audit, and improve autonomous operations safely.

Description:

The overlay should capture evidence at every major step: request, reasoning, summary, data sources, tool calls, policy checks, approvals, actions, and outcomes. Evidence becomes the foundation for trust, compliance, troubleshooting, and production readiness.

Acceptance criteria:

- Evidence objects are defined for request, recommendation, policy decision, approval tool call, and result
- Evidence records include timestamp, correlation ID, agent identity, user identity, and outcome
- Missing evidence blocks promotion to production workflows
- Evidence can be reviewed by architecture, security and operations teams
- Workflow failures include enough evidence for root-cause analysis

4. USER-xx – Request an agent assisted operational workflow
   Category: User experience / operations

As an AT&T operations user, I want to request an agent-assisted workflow in plan language so that I can complete complex operational tasks without manually navigating multiple systems.

Description

User should be able to describe that they need done, such as summarizing an issue, gathering system context, preparing a remediation plan, or drafting a change request. The agentic overlay should translate the request into governed steps.

Acceptance criteria

- User can submit natural language task requests
- The overlay identifies required systems, tools and approvals
- The agent presents a proposed action plan before high-impact execution
- User can approve, reject, or modify the proposed plan
- Completed workflow includes a summary and evidence trail.

5. USER-xx – Review and approve agent recommended actions
   Category: User / Governance

As an AT&T approver, I want to review agent recommended actions before execution so that I can maintain control over sensitive operational decisions

Description

For workflows involving production changes, customer impact, privileged access, or compliance sensitive actions, the agent should stop and present a clear recommendation with rationale, risk, evidence and rollback plan

Acceptance criteria

- Agent identifies actions requiring approval
- Recommended actions include rationale, affected systems, expected impact, risk level, and rollback plan
- Approver can approve, reject, request changes, or escalate
- Approval decision is logged with timestamp and approver identity
- Agent cannot execute approval required actions without explicit approval

6. USER-xx – Receive and agent generated work summary after task completion
   Category: user experience

As an AT&T team lead, I want a clear summary of completed agentic workflows so that I can understand what happened, what changed what was blocked and what needs follow-up

Description

After an agentic workflow is complete, the overlay should produce a concise operational summary suitable for team review, audit, and handoff. This reduces the burden on manual reporting and improves transparency

Acceptance criteria

- Summary includes request, actions taken, systems touched, approvals, blockers, and final outcome.
- Any failed or skipped steps are clearly identified
- Follow-up actions are assigned or queued
- Evidence links or correlation ID is included
- Summary is readable by both technical and non-technical stakeholders

Open Items

- Sandbox – Docker image

- Agentic supervisor/detection
- Business use case development
- Directional integration with Agentic overlay
- Safe resource repository – vetted Skills library, MCP repository
- Network sniffer, Wireshark
- SQL Lite usage, credential storage
- Microsoft 365 Security layer telemetry